



ANCIENT CIVILIZATION RECONSTRUCTION & HISTORICAL INTELLIGENCE ENGINE (ACRHIE)

3.2 Student Details

Name: Pallavi Sarovar

Roll Number: 150096725219

Course: Data Structures and Algorithms – I Using C++

Ancient Civilization Reconstruction & Historical Intelligence Engine (ACRHIE)

The project is a C++ based archaeological data management and analysis system that stores, processes, and analyzes artifact information discovered from ancient civilizations. The system demonstrates the practical implementation of various Data Structures and Algorithms including Sorting, Searching, Hashing, Stack, Queue, Binary Search Tree (BST), Graph Traversals (BFS and DFS), and Dijkstra's Algorithm.

3.4 Problem Statement

Archaeologists frequently discover large numbers of artifacts belonging to different ancient civilizations. Managing these artifacts efficiently and analyzing relationships between civilizations can become challenging when the volume of historical data increases.

The objective of this project is to develop a system that can:

- Store artifact information.
- Search and retrieve artifact records efficiently.
- Rank artifacts based on historical significance.
- Maintain excavation reports.
- Restore previous artifact classifications.
- Represent relationships between civilizations.
- Analyze trade routes using graph algorithms.

The project demonstrates how Data Structures and Algorithms can be applied to solve real-world historical data management problems.

3.5 Objectives

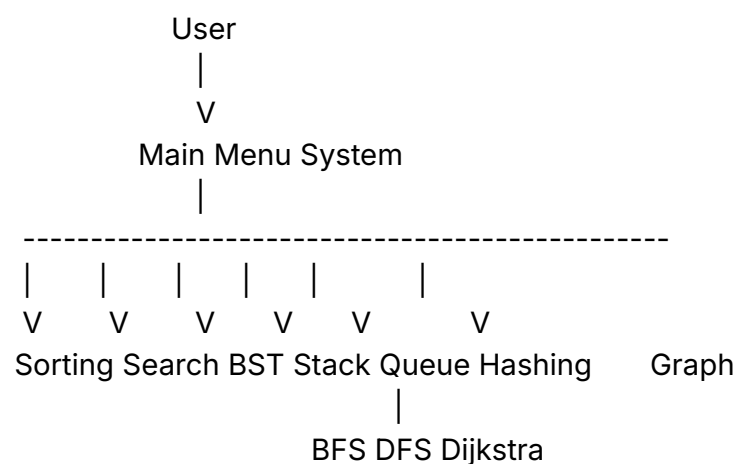
The primary objectives of this project are:

1. To store and manage artifact records efficiently.
2. To implement sorting techniques for ranking artifacts.
3. To implement searching techniques for fast retrieval of records.
4. To utilize hashing for quick artifact identification.
5. To demonstrate Stack operations through undo functionality.

6. To demonstrate Queue operations through excavation report processing.
 7. To implement Binary Search Tree (BST) for organized artifact storage.
 8. To represent civilization relationships using Graphs.
 9. To perform BFS and DFS traversals on civilization networks.
 10. To find shortest trade routes using Dijkstra's Algorithm.
-

3.6 Design / Architecture

System Architecture



Modules

Artifact Management Module

- Add Artifact
- Display Artifact
- Search Artifact
- Sort Artifact

Data Structure Module

- Hash Table
- Stack
- Queue
- Binary Search Tree

Graph Analysis Module

- BFS Traversal
 - DFS Traversal
 - Dijkstra Algorithm
-

3.7 Algorithm Description

1. Sorting Algorithm

Artifacts are sorted based on their historical importance score in descending order.

Purpose: Rank artifacts according to significance.

Steps:

1. Read all artifact records.
 2. Compare importance scores.
 3. Arrange records in descending order.
 4. Display sorted records.
-

2. Binary Search

Binary Search is used to locate an artifact by its ID.

Steps:

1. Sort artifacts according to ID.
 2. Compare target ID with middle element.
 3. Move left or right depending on comparison.
 4. Repeat until found.
-

3. Hashing

Hashing is implemented using `unordered_map`.

Purpose:

Provides nearly constant time lookup of artifact records.

4. Stack

Stack follows the LIFO (Last In First Out) principle.

Purpose:

Restore previous artifact classifications using Undo functionality.

Operations:

- Push
 - Pop
 - Top
-

5. Queue

Queue follows the FIFO (First In First Out) principle.

Purpose:

Process excavation reports in the order they arrive.

Operations:

- Push
 - Front
 - Pop
-

6. Binary Search Tree (BST)

BST stores artifact records according to Artifact ID.

Operations:

- Insert
- Search

- Inorder Traversal
-

7. Breadth First Search (BFS)

BFS explores neighboring civilizations level by level.

Purpose:

Analyze civilization connections.

8. Depth First Search (DFS)

DFS explores civilizations deeply before backtracking.

Purpose:

Analyze historical relationship paths.

9. Dijkstra's Algorithm

Dijkstra's Algorithm finds the shortest trade route between civilizations.

Purpose:

Analyze trade network efficiency.

3.8 Complexity Analysis

Operation	Algorithm	Time Complexity
Search Artifact	Binary Search	$O(\log n)$
Sort	Sorting	$O(n \log n)$

Artifacts		
Hash Search	Hashing	$O(1)$
Stack Operations	Stack	$O(1)$
Queue Operations	Queue	$O(1)$
BST Search	BST	$O(\log n)$
BFS Traversal	Graph	$O(V + E)$
DFS Traversal	Graph	$O(V + E)$
Dijkstra Algorithm	Graph	$O((V+E) \log V)$

3.9 Screenshots of Execution

```
pallavisarovar@pallavis-MacBook-Air Acrhie % g++ -std=c++11 src/main.cpp src/Engine.cpp -o acrhie
pallavisarovar@pallavis-MacBook-Air Acrhie % ./acrhie
Loading Data...
Artifact added: Rosetta Stone (ID: 101)
Artifact added: Terracotta Army (ID: 102)
Artifact added: Dead Sea Scrolls (ID: 103)
Artifact added: Antikythera Mechanism (ID: 104)
Artifact added: Code of Hammurabi (ID: 105)
Trade route added between Egyptian and Greek (Dist: 800)
Trade route added between Egyptian and Babylonian (Dist: 1200)
Trade route added between Greek and Babylonian (Dist: 1500)
Trade route added between Babylonian and Indian (Dist: 2500)
Trade route added between Indian and Chinese (Dist: 3000)
Report 1 added to queue.
Report 2 added to queue.
Data successfully loaded from data/sample_input.txt
```

```
=====
Ancient Civilization Reconstruction Engine (ACRHIE)
=====
1. View Top Significant Artifacts (Sorting)
2. View Artifacts Timeline (BST)
3. Search Artifact by ID (Hashing)
4. Search Artifact by ID (Binary Search)
5. Reclassify Artifact
6. Undo Reclassification (Stack)
7. Process Excavation Report (Queue)
8. Analyze Civilization Trade Network (BFS)
9. Analyze Civilization Trade Network (DFS)
10. Find Shortest Trade Route (Dijkstra's)
11. Exit
Select an option: 1
--- Top Artifacts by Significance ---
1. Rosetta Stone (Score: 99) - Egyptian
2. Dead Sea Scrolls (Score: 98) - Jewish
3. Code of Hammurabi (Score: 97) - Babylonian
4. Antikythera Mechanism (Score: 96) - Greek
5. Terracotta Army (Score: 95) - Chinese
```

```
=====
Ancient Civilization Reconstruction Engine (ACRHIE)
=====
1. View Top Significant Artifacts (Sorting)
2. View Artifacts Timeline (BST)
3. Search Artifact by ID (Hashing)
4. Search Artifact by ID (Binary Search)
5. Reclassify Artifact
6. Undo Reclassification (Stack)
7. Process Excavation Report (Queue)
8. Analyze Civilization Trade Network (BFS)
9. Analyze Civilization Trade Network (DFS)
10. Find Shortest Trade Route (Dijkstra's)
11. Exit
Select an option: 2
--- Artifacts Sorted by Age (Oldest to Newest) ---
Age: -1750 | Name: Code of Hammurabi | Civ: Babylonian
Age: -300 | Name: Dead Sea Scrolls | Civ: Jewish
Age: -210 | Name: Terracotta Army | Civ: Chinese
Age: -196 | Name: Rosetta Stone | Civ: Egyptian
Age: -100 | Name: Antikythera Mechanism | Civ: Greek
```

```
=====
Ancient Civilization Reconstruction Engine (ACRHIE)
=====
1. View Top Significant Artifacts (Sorting)
2. View Artifacts Timeline (BST)
3. Search Artifact by ID (Hashing)
4. Search Artifact by ID (Binary Search)
5. Reclassify Artifact
6. Undo Reclassification (Stack)
7. Process Excavation Report (Queue)
8. Analyze Civilization Trade Network (BFS)
9. Analyze Civilization Trade Network (DFS)
10. Find Shortest Trade Route (Dijkstra's)
11. Exit
Select an option: 3
Enter Artifact ID: 102
Found Artifact via Hash Table: Terracotta Army (Civ: Chinese)
```



```
=====
Ancient Civilization Reconstruction Engine (ACRHIE)
=====
1. View Top Significant Artifacts (Sorting)
2. View Artifacts Timeline (BST)
3. Search Artifact by ID (Hashing)
4. Search Artifact by ID (Binary Search)
5. Reclassify Artifact
6. Undo Reclassification (Stack)
7. Process Excavation Report (Queue)
8. Analyze Civilization Trade Network (BFS)
9. Analyze Civilization Trade Network (DFS)
10. Find Shortest Trade Route (Dijkstra's)
11. Exit
Select an option: 4
Enter Artifact ID: 104
Found Artifact via Binary Search: Antikythera Mechanism (Civ: Greek)
```

```
=====
Ancient Civilization Reconstruction Engine (ACRHIE)
=====
1. View Top Significant Artifacts (Sorting)
2. View Artifacts Timeline (BST)
3. Search Artifact by ID (Hashing)
4. Search Artifact by ID (Binary Search)
5. Reclassify Artifact
6. Undo Reclassification (Stack)
7. Process Excavation Report (Queue)
8. Analyze Civilization Trade Network (BFS)
9. Analyze Civilization Trade Network (DFS)
10. Find Shortest Trade Route (Dijkstra's)
11. Exit
Select an option: 5
Enter Artifact ID to reclassify: 103
Enter new civilization: Chinese
Artifact 103 reclassified to Chinese
```

```
=====
Ancient Civilization Reconstruction Engine (ACRHIE)
=====
1. View Top Significant Artifacts (Sorting)
2. View Artifacts Timeline (BST)
3. Search Artifact by ID (Hashing)
4. Search Artifact by ID (Binary Search)
5. Reclassify Artifact
6. Undo Reclassification (Stack)
7. Process Excavation Report (Queue)
8. Analyze Civilization Trade Network (BFS)
9. Analyze Civilization Trade Network (DFS)
10. Find Shortest Trade Route (Dijkstra's)
11. Exit
Select an option: 6
Undo successful: Artifact 103 restored to Jewish
```

```
=====
Ancient Civilization Reconstruction Engine (ACRHIE)
=====
1. View Top Significant Artifacts (Sorting)
2. View Artifacts Timeline (BST)
3. Search Artifact by ID (Hashing)
4. Search Artifact by ID (Binary Search)
5. Reclassify Artifact
6. Undo Reclassification (Stack)
7. Process Excavation Report (Queue)
8. Analyze Civilization Trade Network (BFS)
9. Analyze Civilization Trade Network (DFS)
10. Find Shortest Trade Route (Dijkstra's)
11. Exit
Select an option: 7
Processing Report ID: 1
Site: Valley of Kings | Date: 2023-01-15
Summary: Uncovered new tomb annex (3 artifacts)
```

```

=====
Ancient Civilization Reconstruction Engine (ACRHIE)
=====
1. View Top Significant Artifacts (Sorting)
2. View Artifacts Timeline (BST)
3. Search Artifact by ID (Hashing)
4. Search Artifact by ID (Binary Search)
5. Reclassify Artifact
6. Undo Reclassification (Stack)
7. Process Excavation Report (Queue)
8. Analyze Civilization Trade Network (BFS)
9. Analyze Civilization Trade Network (DFS)
10. Find Shortest Trade Route (Dijkstra's)
11. Exit
Select an option: 8
Enter Civilization Name (e.g., Egyptian): Indian
Analyzing connections for Indian...
BFS Traversal from Indian: Indian Babylonian Chinese Egyptian Greek

```

```

=====
Ancient Civilization Reconstruction Engine (ACRHIE)
=====
1. View Top Significant Artifacts (Sorting)
2. View Artifacts Timeline (BST)
3. Search Artifact by ID (Hashing)
4. Search Artifact by ID (Binary Search)
5. Reclassify Artifact
6. Undo Reclassification (Stack)
7. Process Excavation Report (Queue)
8. Analyze Civilization Trade Network (BFS)
9. Analyze Civilization Trade Network (DFS)
10. Find Shortest Trade Route (Dijkstra's)
11. Exit
Select an option: 9
Enter Civilization Name (e.g., Egyptian): Greek
Analyzing connections for Greek using DFS...
DFS Traversal from Greek: Greek Egyptian Babylonian Indian Chinese

```

```

=====
Ancient Civilization Reconstruction Engine (ACRHIE)
=====
1. View Top Significant Artifacts (Sorting)
2. View Artifacts Timeline (BST)
3. Search Artifact by ID (Hashing)
4. Search Artifact by ID (Binary Search)
5. Reclassify Artifact
6. Undo Reclassification (Stack)
7. Process Excavation Report (Queue)
8. Analyze Civilization Trade Network (BFS)
9. Analyze Civilization Trade Network (DFS)
10. Find Shortest Trade Route (Dijkstra's)
11. Exit
Select an option: 10
Enter Start Civilization: Chinese
Enter Destination Civilization: Greek
Shortest route distance: 7000
Path: Chinese -> Indian -> Babylonian -> Greek

```

```

=====
Ancient Civilization Reconstruction Engine (ACRHIE)
=====
1. View Top Significant Artifacts (Sorting)
2. View Artifacts Timeline (BST)
3. Search Artifact by ID (Hashing)
4. Search Artifact by ID (Binary Search)
5. Reclassify Artifact
6. Undo Reclassification (Stack)
7. Process Excavation Report (Queue)
8. Analyze Civilization Trade Network (BFS)
9. Analyze Civilization Trade Network (DFS)
10. Find Shortest Trade Route (Dijkstra's)
11. Exit
Select an option: 11
Exiting system. Goodbye!
pallavisarovar@pallavis-MacBook-Air Acrhie %

```

3.10 Results and Findings

The project successfully demonstrates the application of multiple Data Structures and Algorithms in a real-world archaeological data management scenario.

The system efficiently stores artifact records, performs fast searches, sorts artifacts according to importance, processes excavation reports, restores previous classifications, and analyzes civilization networks.

Graph algorithms successfully identify civilization relationships and shortest trade routes.

3.11 Conclusion

The Ancient Civilization Reconstruction & Historical Intelligence Engine (ACRHIE) successfully integrates various Data Structures and Algorithms into a single application.

The project demonstrates practical applications of:

- Sorting
- Searching
- Hashing
- Stack
- Queue
- Binary Search Tree
- BFS
- DFS
- Dijkstra's Algorithm

The implementation highlights how DSA concepts can be applied to efficiently manage and analyze historical information systems.

3.12 GitHub Repository Link

GitHub Repository:

<https://github.com/pallavi-676/ACRHIE-caseStudy>

